04 / PSEUDOCODE GUIDE

From geometry to an algorithm

EX30 · Planning before programming

Goal. Convert a physical operation into an ordered numerical procedure before writing Python. Validation helps detect sign, axis and matrix-order mistakes.

1. From the physical problem to inputs and outputs

Before calculating, identify what changes and what remains fixed.

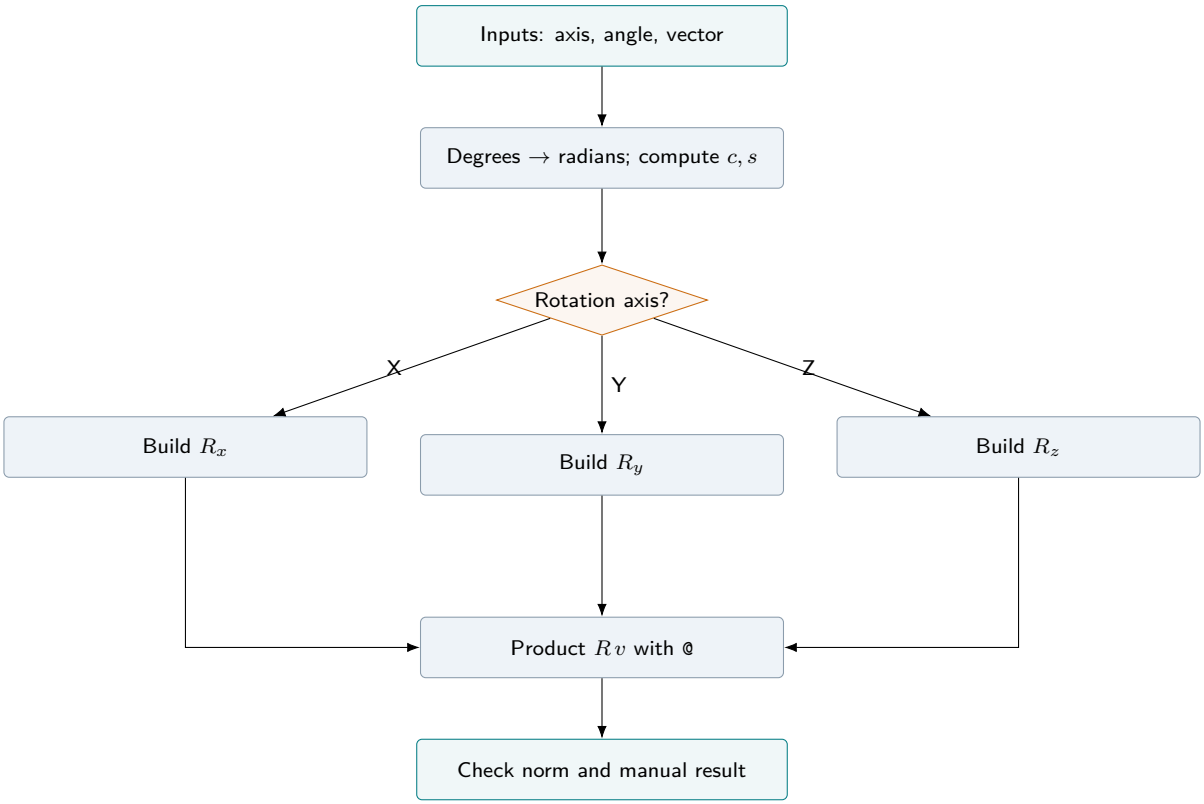
1. **Operation:** does the physical vector rotate, or do the axes used to describe it change?
2. **Inputs:** rotation axis (X, Y or Z), angle in degrees, three-component vector and optionally a lever arm.
3. **Outputs:** rotated vector or coordinates in another frame, norm, combined matrix or torque.
4. **Validation:** worked mathematical result, norm, orthogonality, inverse transformation.

2. From algebra to an algorithm: building a 3D rotation

1. Read the angle θ and selected axis.
2. Convert θ from degrees to radians.
3. Compute $c = \cos\theta$ and $s = \sin\theta$.
4. For that axis, keep its associated component fixed with a 1; place a 2D rotation block on the other two coordinates.
5. Verify that the resulting matrix has shape `(3,3)`.
6. Apply the matrix-vector product and compare against a hand calculation.

Do not change an angle variable and reuse a matrix that was already built for a previous angle: rebuild it.

3. Visual decision flow



4. Guided pseudocode — one active rotation

```
START
    choose an axis and angle_in_degrees
    convert angle to radians
    compute cosine and sine
    build rotation matrix R for that axis
    define initial vector v
    compute rotated_vector = R @ v
    check shape, expected result and norm
END
```

There is no need yet to decide on a function design; following the mathematical flow is more important.

5. Pseudocode — two turns in a defined order

```
START
    build R_first for the first axis and angle
    build R_second for the second axis and angle
    compute intermediate = R_first @ vector
    compute final = R_second @ intermediate
    repeat with the opposite rotation order
    compare both vectors (allclose)
    verify norms
END
```

Pitfall: inadvertently reversing the multiplication order. In 3D, results can differ even though both final vector norms remain unchanged.

6. Pseudocode — engineering: force and torque

1. Define the initial force in newtons and build the two required matrices.
2. Apply the rotations in the physically specified order; verify force magnitude.
3. Define the lever arm in metres and calculate torque with a cross product (EX27).
4. Report final force (N), torque (N m), and interpret the signs.

7. Pseudocode — passive reference-frame change

1. Specify global components of a physically fixed force.
2. Build the matrix whose columns are the local axes expressed in global coordinates.
3. Compute local components using `R.T @ F_global`.
4. Reconstruct global components with `R @ F_local`; verify the complete vector.
5. Verify the norm without confusing norm equality with component equality.

8. Self-check before programming

1. Can I identify the component preserved by each axis rotation?
2. Do all matrix signs follow the right-hand rule?
3. Can I explain why `R2 @ R1 @ v` applies `R1` first?
4. Am I using `np.allclose` for approximate floating-point comparisons?
5. Can I distinguish a rotated physical force from components of a fixed force in rotated local axes?

Final idea: construct → apply → validate → interpret. In this exercise, understanding axes and rotation order matters more than minimising code length.